CartPole DQN Extension — Algorithms, Pseudocode, Metrics & Setup

Reference: Sun & Zheng (ICLR 2024) — *Belief-Enriched Pessimistic Q-Learning Against Adversarial State Perturbations* **Code:** `main_cartpole_dqn.py` (currently shipped as `main_cartpole_dqn.txt`), `cartpole_belief.py` **Environment:** CartPole-v1 (Gymnasium)

---

1. The three algorithms (plain language + technical detail)

Naming note. All three methods use Q-learning / DQN and are therefore *off-policy* in the classical RL sense (the behavior that generates data need not match the greedy target policy). In prior emails we called the third method "off-policy DQN," which is misleading. Below we rename it **Pessimistic-Behavior / Greedy-Target DQN (PB-GT DQN)**.

Xiaolin's paper presents **two** deep pessimistic variants:

Paper variant	Algorithms	Belief mechanism
BP-DQN	Alg. 4 (train), Alg. 5 (test)	PF-RNN particles from observation–action history
DP-DQN	Alg. 6–7	Diffusion purification of pixel observations

Our CartPole pessimistic method is a **BP-DQN-style** port (maximin over a belief set + pessimistic Bellman target), **not** DP-DQN. Belief update **is included** in the fair 5-seed runs (`belief_mode=learned` in `cartpole_experiments_fair_v2.csv`), via a pretrained GRU belief RNN that emits κ particles in $B_\varepsilon(\tilde{s})$. This is a simplified PF-RNN analogue, not the exact particle-filter RNN of Ma et al. (2020) used in Xiaolin's continuous Gridworld.

1.1 Vanilla Double DQN

Idea. Standard deep Q-learning with no notion of observation attack. The agent always sees the true (normalized) CartPole state.

Train. ε-greedy exploration on the true state s. Transitions (s, a, r, s') go into a replay buffer. Updates use Double DQN / Huber loss:

$$y = r + \gamma(1 - \text{done})Q_{\theta'}(s', \arg\max_{a'} Q_\theta(s', a')).$$

Test (clean or under attack). Greedy action on what the defender observes:

$$a = \arg\max_a Q_\theta(s_{\text{obs}}, a),$$

where $s_{\text{obs}} = s$ if clean and $s_{\text{obs}} = \tilde{s}$ under attack. There is **no** belief and **no** maximin.

CartPole specifics. MLP Q with hidden layers `[128,128]`; observations linearly scaled to roughly $[-1, 1]$; 500k environment steps from scratch (with a solve-gate retry if clean eval < 475).

1.2 Belief-enriched pessimistic DQN (BP-DQN-style)

Idea. The defender never trusts a single observation. Given a (possibly perturbed) observation $\tilde{s}$, it forms a **belief** M — a finite set of candidate true states — and chooses the action that looks best in the *worst* candidate (maximin). The same principle is baked into the Bellman target so training and testing are consistent.

Which paper variant? Closest to **BP-DQN** (Algorithms 4–5): belief particles + maximin action + maximin TD target, fine-tuned from a pretrained vanilla DQN. We do **not** use DP-DQN / diffusion (inappropriate for low-dimensional CartPole, and unused in our code).

Belief update (included). Before robust fine-tuning we pretrain a small **BeliefRNN** (GRU + particle head) for 30k steps on clean vanilla rollouts with synthetic L_∞ noise. At each defender step:

1. Append the current observation to the recurrent state.
2. Emit κ = 20 particle offsets via $\tanh(\cdot) \cdot \varepsilon$, clipped into valid CartPole bounds.
3. Use that particle set as M_t.

Legacy fallback (ablation): i.i.d. uniform samples in $B_\varepsilon(\tilde{s})$ with **no** history (*UniformBallBelief*). Fair reported runs used `learned` belief.

Train (after cloning vanilla weights).

1. Soften training: freeze MLP backbone, train only the final linear head; lower LR and ε.

2. At each step, form a worst-case observation $\tilde{s}$ via random-search PGD (32 candidates in the L_∞ ball, score $Q(s, \pi(\tilde{s}))$).
3. Behavior policy: ε-greedy over the **maximin** action

$$\pi(\tilde{s}) = \arg\max_a \min_{m \in M} Q_\theta(m, a).$$

1. Store true transition plus next belief (s, a, r, s', M') in a belief replay buffer.
2. TD target (Algorithm 4, line 13):

$$y = r + \gamma(1 - \text{done}) \max_{a'} \min_{m \in M'} Q_{\theta'}(m, a').$$

1. Mid-training clean eval every 25k steps; keep best checkpoint; early-stop if the policy collapses after reaching ≥475.

Test under attack. Maximin over the belief centered at the attacker's $\tilde{s}$ (Algorithm 5 style). Clean tables also report *greedy on true s* for comparability with GridWorld's clean column.

1.3 Pessimistic-behavior / greedy-target DQN (PB-GT DQN)

(formerly "off-policy DQN" / "off-policy greedy target")

Idea. Keep the **robust data-collection behavior** of pessimistic DQN (act with maximin under a PGD-perturbed observation, maintain beliefs while rolling out), but **change the learning target** to ordinary greedy bootstrap on the *true* next state — the same target vanilla DQN uses. At test time, drop belief/maximin entirely and act greedily on the observed state. This is the deep analogue of our tabular GridWorld extension (Eq. 5.1–5.2 in our annotated GridWorld code): a deliberate **behavior–target mismatch** designed for faster inference while still seeing pessimistic rollouts during training.

Why not call it "off-policy DQN"? Vanilla and pessimistic DQN are also off-policy. The distinctive claim is the *pairing* of pessimistic behavior with a non-pessimistic (greedy) target and greedy test policy.

Train. Same collection loop as pessimistic (PGD $\tilde{s}$, ε-greedy maximin, belief particles stored). Update rule uses Double DQN greedy targets on true s', **ignoring** M' in the backup:

$$y = r + \gamma(1 - \text{done})Q_{\theta'}(s', \arg\max_{a'} Q_\theta(s', a')).$$

Default fine-tune: 40k steps, LR 10^{-5}, ε start 0.01 (shorter/gentler than pessimistic's 100k / $5 \cdot 10^{-5}$).

Test. Identical to vanilla: $a = \arg\max_a Q(\tilde{s}, a)$ under attack; no belief, no maximin → ~50× faster than pessimistic under matched attack in our runs.

Comparison snapshot

	Vanilla DQN	Pessimistic (BP-style)	PB-GT DQN (3rd method)
Behavior at train	ε-greedy on true s	ε-greedy **maximin** on PGD $\tilde{s}$	same as pessimistic
Bellman target	greedy on true s'	$\max\min$ over next belief M'	greedy on true s'
Test under attack	greedy on $\tilde{s}$	**maximin** over belief	greedy on $\tilde{s}$
Belief update	none	yes (learned GRU / optional uniform ball)	collected but unused in target/test
Paper counterpart	vanilla DQN	BP-DQN (simplified)	*ours* (not in Xiaolin's paper)

---

2. Pseudocode for PB-GT DQN (Algorithm-4 style)

Format mirrors Appendix E, Algorithm 4 (BP-DQN). Differences from Algorithm 4 are highlighted in **bold**.

Algorithm A: Pessimistic-Behavior / Greedy-Target DQN (PB-GT DQN) — Training

Data: Number of iterations T; pretrained vanilla Q-network Q_v; belief model N_p (GRU particle belief); target-update frequency Z; batch size D; exploration parameter ε'; attack budget ε; candidate count K_{PGD}

Result: Fine-tuned Q-network Q_g

1. Initialize replay buffer $\mathcal{B}$; set $Q_g \leftarrow Q_v$, target $Q' \leftarrow Q_v$; initialize observation/action history for N_p
2. **Optionally freeze all layers of Q_g except the final linear head; set a small learning rate**
3. **for** $t = 0, 1, \ldots, T$ **do**
4. Use random-search PGD (K_{PGD} candidates) to find $\tilde{s}_t \in B_\varepsilon(s_t)$ minimizing $Q_g(s_t, \pi(\tilde{s}_t))$, where π is greedy w.r.t. Q_g
5. Form belief M_t from N_p (or intersect / sample in $B_\varepsilon(\tilde{s}_t)$)
6. Choose action with ε-greedy **maximin** behavior:

$a_t = \arg\max_a \min_{m \in M_t} Q_g(m, a)$ with probability $1 - \varepsilon'$; else a_t random

1. Append $\tilde{s}_t, a_t$ to history; update N_p to obtain next belief M_{t+1}
2. Execute a_t; observe reward R_t and **true** next state s_{t+1}
3. **if** s_{t+1} terminal **then** reset belief history
4. Store transition $s_t, a_t, R_t, s_{t+1}, M_{t+1}$ in $\mathcal{B}$

*(belief stored for logging / parity with Alg. 4; **not** used in the target below)*

1. Sample minibatch $s_i, a_i, R_i, s_{i+1}, M_{i+1}$ of size D from $\mathcal{B}$
2. **Set greedy (vanilla-style) targets — the key difference from Algorithm 4, line 13:**

$$y_i = \begin{cases} R_i & \text{if } s_{i+1} \text{ terminal} \\ R_i + \gamma \max_{a'} Q'(s_{i+1}, a') & \text{otherwise} \end{cases}$$

(Double DQN in code: $a' = \arg\max Q_g(s_{i+1}, \cdot)$, evaluated with Q') **Do not** use $\max_{a'} \min_{m \in M_{i+1}} Q'(m, a')$

1. Gradient step on Huber loss $\sum_i (y_i - Q_g(s_i, a_i))^2$ (smooth L_1)
2. Update target network every Z steps (in our CartPole code: every Z *episodes*)
3. **end for**

Algorithm B: PB-GT DQN — Testing

Data: Trained Q_g

1. **for** each step t **do**
2. Observe (possibly perturbed) state $\tilde{s}_t$
3. **Select greedy action** $a_t = \arg\max_a Q_g(\tilde{s}_t, a)$

(No belief M_t, no maximin — difference from Algorithm 5)

1. Execute a_t
2. **end for**

Relation to Algorithm 4. Collection (lines 4–8) matches BP-DQN's pessimistic ε-greedy maximin behavior under PGD observations. The backup (line 12) and the test policy (Algorithm B) revert to vanilla greedy. Belief is maintained during training for pipeline uniformity but is intentionally unused in y_i and at test time.

---

3. Evaluation metrics, hardware, and hyperparameters

3.1 The four reported CartPole metrics

All scores are **mean episodic return** over 100 evaluation episodes per seed (CartPole-v1 capped at 500), then averaged across seeds. Observation perturbations live in an L_∞ ball of radius $\varepsilon = 0.05$ in *raw* CartPole coordinates, then re-normalized.

(1) Clean

No attacker. Defender observes the true state.

- Vanilla & PB-GT: greedy on s.
- Pessimistic: we report **greedy on s** as the primary clean number (GridWorld parity); CSV also logs maximin-on-clean as `pessimistic_test_maximin_`*.

(2) Matched attacker

Per-defender learned adversary. For each seed we train a separate Stable-Baselines3 DQN attacker *against that defender*, then evaluate that defender against *its own* attacker.

- Attacker action space: discrete $3^4 = 81$ choices — each obs dimension gets $\delta_i \in -\varepsilon, 0, +\varepsilon$.
- Attacker reward = –defender reward; attacker sees the true state, applies δ, defender sees $\tilde{s} = \mathrm{clip}(s + \delta)$.
- Vanilla / PB-GT defend with greedy on $\tilde{s}$; pessimistic defends with **maximin over belief** at $\tilde{s}$.
- Attackers are trained for 80k steps after defenders are frozen.

This answers: "How robust is each method against an adversary specialized to *it*?" It is **not** the same threat for all three methods (attack strength can differ).

(3) Shared random δ

Identical weak threat distribution across defenders. For each $(\mathrm{seed}, \mathrm{episode}, \mathrm{step})$ we draw one $\delta \sim \mathrm{Unif}([-\varepsilon, \varepsilon]^4)$ from a shared RNG stream and apply it to that defender's true state. Every method faces the *same* random perturbation process (though trajectories diverge after the first differing action).

This answers: "Under a fixed, non-adaptive noise model, how do the three policies compare?" It removes confounding from matched-attacker specialization.

(4) Worst-case (PGD*)

Defender-specific approximate worst-case search (not a single shared PGD path). At each step, sample 32 random δ in the L_∞ ball and pick

$$\tilde{s}^\star = \arg\min_{\tilde{s} \in B_\varepsilon(s)} Q\big(s, \pi(\tilde{s})\big)$$

with π = greedy for vanilla/PB-GT and maximin for pessimistic. Then the defender acts on $\tilde{s}^\star$ with its usual test policy.

Caveat (important for the paper). This is *not* shared across algorithms the way Xiaolin's Appendix F PGD eval is; each defender's Q-landscape defines its own "worst" $\tilde{s}$. In tables we mark it **PGD** / "defender-specific worst-case (32 candidates)."

Inference timing. Under matched attack we also record ms/step. Pessimistic maximin was ~2.0 ms/step vs ~0.04 ms/step for greedy (≈52×), analogous in spirit to GridWorld's robustness–speed tradeoff.

3.2 Hardware and software (Appendix F.2 style)

Item	Configuration used for CartPole fair 5-seed runs
Machine	Apple MacBook Pro (M1 Pro), 16 GB unified memory
Accelerator	Apple Silicon (CPU / MPS available); runs used `device=cpu` in `DQNConfig`
OS	macOS 15.7.7 (arm64)
Python	3.9.13
PyTorch	2.8.0 (CUDA unavailable on this machine)
Gymnasium	1.1.1 (`CartPole-v1`)
Stable-Baselines3	2.7.0 (attacker DQN only)
Training budget unit	Environment **steps** (not Atari frames); CartPole episodes ≤ 500 steps

Note for arXiv. Xiaolin's F.2 used an i9-12900KF + RTX 3090. Our CartPole numbers were produced on CPU Apple Silicon. For the requested Pong/Freeway reproduction we should use a GPU workstation (as offered) and restate hardware there.

3.3 Hyperparameters (CartPole)

Environment / attack

Parameter	Value
Environment	CartPole-v1
Observation preprocessing	Linear normalize each dim to $[-1, 1]$ using `[±4.8, ±5, ±0.418, ±5]`
Action space	Discrete(2)
Attack budget ε	0.05 (raw units, L_∞)
PGD candidates (train + PGD eval)	32 random δ
Attacker discrete grid	81 actions (3^4)
Eval episodes per metric	100
Seeds (fair table)	5 (`0`–`4`)

Vanilla Double DQN

Parameter	Value
Network	MLP `[128, 128]`, ReLU
Optimizer	Adam, lr 10^{-3}
Discount γ	0.99
Replay buffer	100,000
Batch size	128
Learning starts	1,000
Train frequency	every 4 steps
Target update	every 10 episodes (hard copy)
ε schedule	start 1.0 → end 0.01, exponential decay over 50k steps
Loss	Huber (`smooth_L1`)
Gradient clip	max-norm 10
Training steps	500,000 (+ up to 2×200k continue if clean eval < 475)

Belief model (BP-style)

Parameter	Value
Architecture	GRU (hidden 64) + particle head → κ×4 deltas + next-obs head
κ (belief size)	20 (paper BP-DQN uses κ_p=30 on continuous Gridworld)
Pretrain steps	30,000 on vanilla rollouts + synthetic noise
Pretrain optimizer	Adam, lr 10^{-3}
Fair-run mode	`learned` (GRU); `uniform` available as ablation

Pessimistic fine-tune (BP-style)

Parameter	Value
Init	Clone vanilla weights
Backbone	Frozen; train final linear layer only
LR	$5 \cdot 10^{-5}$
Max fine-tune steps	100,000
ε start / end / decay	0.02 / 0.005 / 15k steps
Mid-eval	every 25k steps (20 episodes greedy + maximin)
Early stop	after best greedy ≥ 475, stop if greedy < 400 or drops > 75 from best
Checkpointing	restore best mid-eval weights at end
TD target	$\max \min$ over next belief (Alg. 4)
Test under attack	maximin over belief (Alg. 5)

PB-GT fine-tune (3rd algorithm)

Parameter	Value
Init	Clone vanilla weights
Backbone	Frozen (via shared `prepare_robust_finetune`)
LR	10^{-5}
Fine-tune steps	**40,000** (shorter; 100k destroyed clean policy in pilots)
ε start / end / decay	0.01 / 0.005 / 10k steps
Behavior	same pessimistic maximin + PGD $\tilde{s}$ as above
TD target	Double DQN **greedy** on true s'
Test	greedy on $\tilde{s}$ (no belief)

Learned attackers (matched metric)

Parameter	Value
Algorithm	SB3 DQN, `MlpPolicy`, net `[128,128]`
Steps	80,000 per (seed, defender)
LR / buffer / batch	10^{-3} / 50k / 128
γ / train_freq / target_update	0.99 / 4 / 500
Exploration	fraction 0.3, final ε 0.05

3.4 Gaps vs Xiaolin Appendix F.2 (to state explicitly)

1. **Environment:** CartPole-v1, not continuous Gridworld or Atari Pong/Freeway.
2. **Belief:** Simplified GRU particle model, not full PF-RNN; κ=20 vs κ_p=30.
3. **PGD:** 32-sample random search, not 10-step iterative PGD with η=10 as in the paper.
4. **Attacks evaluated:** matched SB3 DQN attacker, shared uniform δ, defender-specific PGD — not MinBest / PA-AD.
5. **Hardware / scale:** CPU laptop, n=5 seeds; paper used RTX 3090 and (for Atari) multi-million-frame training.
6. **Vanilla clean variance:** mean 421 ± 176 with seed 2 failing (107) — consistent with the concern that vanilla may not have fully converged on all seeds.